Title: Inter-Process Communication (IPC)

1. AIM

To study and implement Inter-Process Communication (IPC) in Linux using Unix Pipes and Shared Memory. To understand data transmission and synchronization between independent processes using IPC mechanisms.

2. THEORY

2.1 Introduction to Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism provided by an operating system that allows two or more processes to exchange data and coordinate their activities.

In a multitasking operating system, multiple processes execute independently. Each process has its own memory space, so one process cannot directly access the private memory of another process.

IPC provides communication mechanisms that allow processes to exchange information safely and efficiently.

The main purposes of IPC are:

1. Exchange data between processes.
2. Synchronize process execution.
3. Share information between independent processes.
4. Coordinate multiple processes.
5. Improve application efficiency.

2.2 Need for IPC

IPC is required when multiple processes need to work together to complete a particular task.

For example:

- A client process sends a request to a server process.
- A parent process communicates with its child process.
- Multiple processes share common data.
- Applications exchange information between different components.

2.3 Types of IPC Mechanisms

Linux provides several IPC mechanisms.

IPC Mechanism	Description
Pipes	Transfers data between related processes
Named Pipes (FIFO)	Allows communication between unrelated processes
Shared Memory	Allows multiple processes to access a common memory region
Message Queues	Exchanges structured messages between processes
Semaphores	Provides synchronization between processes
Sockets	Enables communication between processes on the same or different systems

2.4 Unix Pipes

A pipe is an IPC mechanism that allows data to flow from one process to another.

A pipe generally provides unidirectional communication, meaning data flows in one direction.

It is commonly used for communication between related processes, such as parent and child processes.

The `pipe()` system call creates a communication channel.

Syntax:

```
None
int pipe(int pipefd[2]);
```

The pipe uses two file descriptors:

- `pipefd[0]`: Read end of the pipe.
- `pipefd[1]`: Write end of the pipe.

Important Functions:

Function	Description
`pipe()`	Creates a pipe
`read()`	Reads data from the pipe
`write()`	Writes data into the pipe
`close()`	Closes a file descriptor
`fork()`	Creates a child process

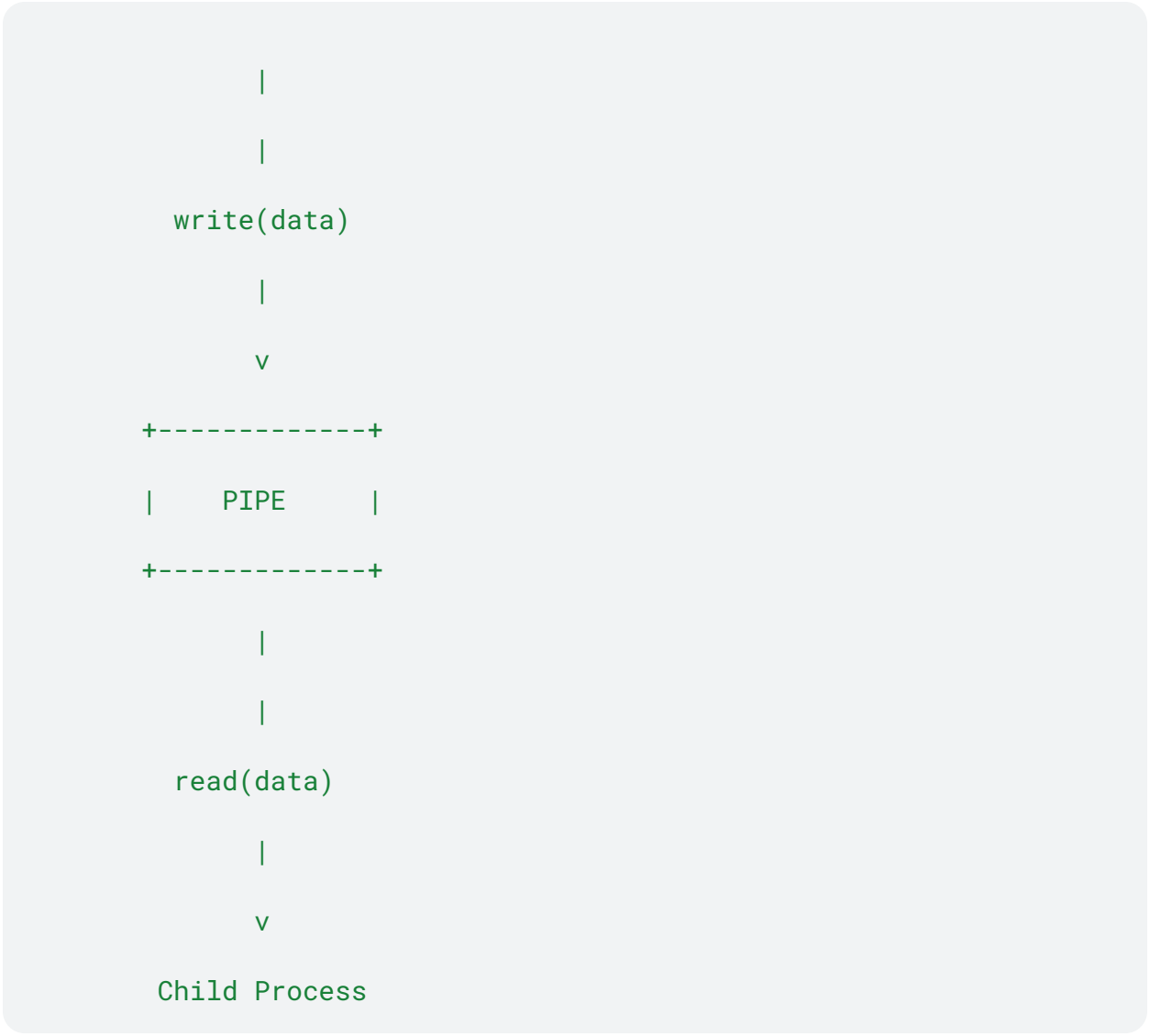
Working of Pipe:

1. The parent process creates a pipe.
2. The parent creates a child process using `fork()`.
3. The parent writes data into the pipe.
4. The child reads data from the pipe.
5. The child displays the received message.

2.5 Pipe Communication Diagram

```
None

       Parent Process
```



2.6 Shared Memory

Shared Memory is one of the fastest IPC mechanisms because multiple processes can access the same memory region.

Instead of transferring data through the kernel for every communication operation, processes directly access a common memory area.

The operating system creates a shared memory segment and allows multiple processes to attach it to their address spaces.

In System V shared memory, the following functions are commonly used.

Function	Description
`shmget()`	Creates or accesses a shared memory segment
`shmat()`	Attaches shared memory to a process
`shmdt()`	Detaches shared memory
`shmctl()`	Controls or removes shared memory

Syntax:

```
None
shmget(key, size, flags);
```

```
None
shmat(shmid, NULL, 0);
```

```
None
shmdt(shared_memory);
```

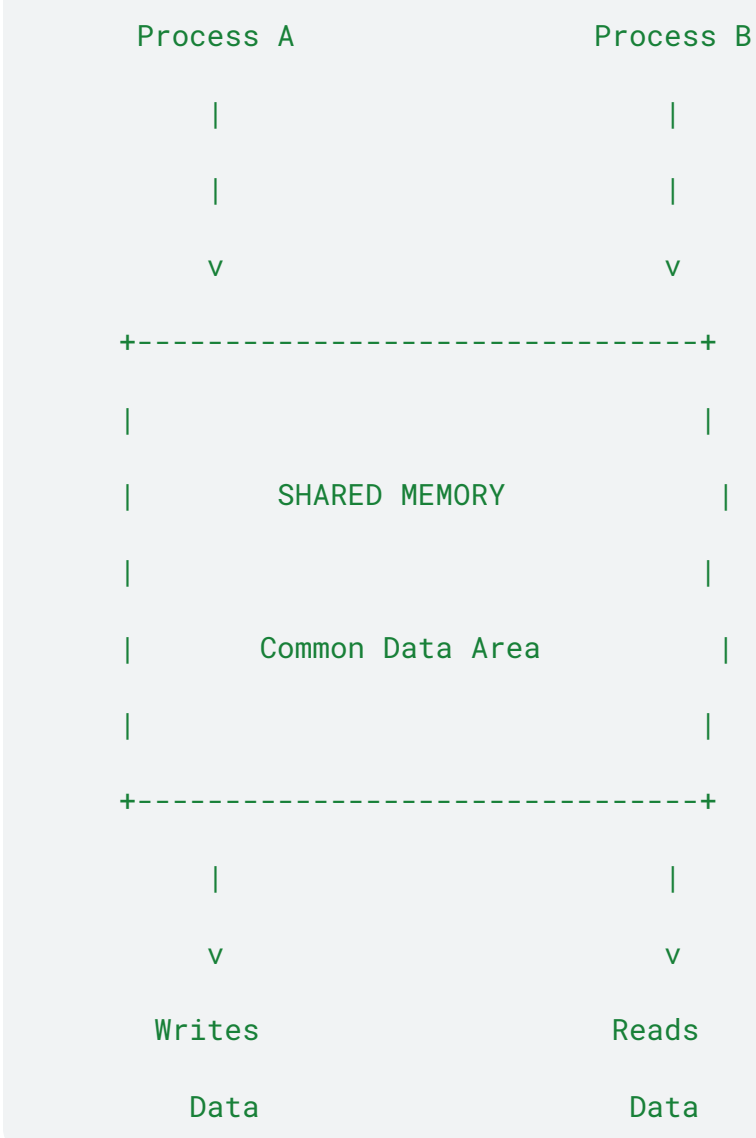
2.7 Working of Shared Memory

1. A process creates a shared memory segment.
2. Another process accesses the same segment.
3. Both processes attach the shared memory.

4. One process writes data into the shared memory.
5. Another process reads the stored data.
6. Both processes detach the memory after communication.
7. The shared memory segment is removed when no longer required.

2.8 Shared Memory Communication Diagram

None



2.9 Difference Between Pipes and Shared Memory

Pipes	Shared Memory

Data is transferred through a communication channel	Processes access a common memory region
Generally unidirectional	Supports access by multiple processes
Commonly used between related processes	Can be used between unrelated processes
Data transfer involves kernel-managed pipe buffers	Data is accessed directly after attachment
Generally simpler to implement	Requires memory management and synchronization
Suitable for small data transfers	Suitable for large data transfers

2.10 Process Synchronization in IPC

Process synchronization ensures that multiple processes access shared resources in a controlled manner.

When two processes access shared data simultaneously, inconsistent results may occur.

Synchronization mechanisms such as semaphores and mutexes help prevent race conditions.

For shared memory, synchronization is particularly important because multiple processes can modify the same memory region.

2.11 Advantages of IPC

1. Enables communication between independent processes.
2. Supports data sharing.
3. Improves coordination between processes.
4. Allows parallel processing.
5. Helps develop client-server applications.
6. Shared memory provides efficient data access.

2.12 Disadvantages of IPC

1. Some IPC mechanisms require complex implementation.
2. Improper synchronization may cause race conditions.
3. Shared memory requires careful resource management.
4. Communication failures may affect application execution.
5. IPC resources must be properly released after use.

3. CODE

This practical demonstrates two IPC mechanisms:

1. Unix Pipes.
2. Shared Memory.

Program 1: Inter-Process Communication Using Unix Pipes

Filename: `pipe.c`

```
None
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <sys/types.h>

#include <sys/wait.h>


int main()

{

    int pipefd[2];

    pid_t pid;
```

```
char message[] = "Hello Child Process!";
char buffer[100];

// Create pipe
if (pipe(pipefd) == -1)
{
    perror("Pipe creation failed");
    exit(1);
}

// Create child process
pid = fork();

if (pid < 0)
{
    perror("Fork failed");
    exit(1);
}

else if (pid > 0)
{
```

```
    // Parent process

    close(pipefd[0]);

    printf("Parent Process Started\n");

    // Write message into pipe
    write(pipefd[1], message, strlen(message) + 1);

    printf("Parent sent: %s\n", message);

    close(pipefd[1]);

    wait(NULL);

    printf("Parent Process Completed\n");
}

else
{
    // Child process
```

```
        close(pipefd[1]);

        printf("\nChild Process Started\n");

        // Read message from pipe
        read(pipefd[0], buffer, sizeof(buffer));

        printf("Child received: %s\n", buffer);

        close(pipefd[0]);

        printf("Child Process Completed\n");

        exit(0);
    }

    return 0;
}
```

Explanation of Program 1

1. Pipe Creation

```
None
pipe(pipefd);
```

Creates a pipe with one reading end and one writing end.

2. Process Creation

```
None
pid = fork();
```

Creates a child process.

3. Parent Process

```
None
write(pipefd[1], message, strlen(message) + 1);
```

The parent process sends a message through the pipe.

4. Child Process

```
None
read(pipefd[0], buffer, sizeof(buffer));
```

The child process receives the message from the pipe.

5. Closing File Descriptors

```
None
close(pipefd[0]);

close(pipefd[1]);
```

Closes the unused or completed pipe ends to release resources.

6. Process Synchronization

```
None
wait(NULL);
```

The parent waits for the child process to finish.

Program 2: Inter-Process Communication Using Shared Memory

Filename: `shared_memory.c`

```
None
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <sys/ipc.h>

#include <sys/shm.h>

#include <sys/wait.h>


int main()

{

    int shmid;


    key_t key = IPC_PRIVATE;


    char *shared_memory;
```

```
// Create shared memory segment
shmid = shmget(key, 1024, IPC_CREAT | 0666);

if (shmid == -1)
{
    perror("Shared memory creation failed");
    exit(1);
}

// Attach shared memory
shared_memory = (char *)shmat(shmid, NULL, 0);

if (shared_memory == (char *)-1)
{
    perror("Shared memory attachment failed");
    shmctl(shmid, IPC_RMID, NULL);
    exit(1);
}

// Create child process
pid_t pid = fork();
```

```
if (pid < 0)
{
    perror("Fork failed");
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    exit(1);
}

else if (pid == 0)
{
    // Child process

    printf("Child Process Started\n");

    // Write data into shared memory
    strcpy(shared_memory,
           "Hello from Child using Shared Memory");

    printf("Child wrote data successfully\n");
```

```
        exit(0);
    }

    else
    {
        // Parent process

        wait(NULL);

        printf("\nParent Process Started\n");

        // Read data from shared memory
        printf("Parent received: %s\n",
               shared_memory);

        // Detach shared memory
        shmdt(shared_memory);

        // Remove shared memory segment
        shmctl(shmid, IPC_RMID, NULL);
```

```
        printf("Shared Memory Released\n");

        printf("Parent Process Completed\n");
    }

    return 0;
}
```

Explanation of Program 2

1. Shared Memory Creation

```
None
shmid = shmget(key, 1024, IPC_CREAT | 0666);
```

Creates a shared memory segment of 1024 bytes.

2. Shared Memory Attachment

```
None
shared_memory = (char *)shmat(shmid, NULL, 0);
```

Attaches the shared memory segment to the process address space.

3. Process Creation

```
None
pid = fork();
```

Creates a child process.

Both parent and child inherit access to the attached shared memory.

4. Writing Data

```
None
strcpy(shared_memory,

       "Hello from Child using Shared Memory");
```

The child process writes a message into the shared memory.

5. Reading Data

```
None
printf("Parent received: %s", shared_memory);
```

The parent process reads the message stored by the child.

6. Waiting for Child

```
None
wait(NULL);
```

The parent waits until the child completes writing the message.

7. Detaching Shared Memory

```
None
shmdt(shared_memory);
```

Detaches the shared memory from the process.

8. Removing Shared Memory

```
None
shmctl(shmid, IPC_RMID, NULL);
```

Marks the shared memory segment for removal.

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.
- Terminal: Linux Terminal.
- Required libraries: POSIX and System V IPC headers.

Step 1: Open Linux Terminal

Press:

```
None
Ctrl + Alt + T
```

Step 2: Check GCC Compiler

```
None
gcc --version
```

If GCC is not installed:

```
None
sudo apt update

sudo apt install build-essential
```

Step 3: Create a Practical Directory

```
None
mkdir IPC_Practical

cd IPC_Practical
```

Step 4: Create the Pipe Program

```
None

nano pipe.c
```

Copy and paste Program 1.

Save the file:

- Press `Ctrl + O`.
- Press Enter.
- Press `Ctrl + X`.

Step 5: Compile the Pipe Program

```
None

gcc pipe.c -o pipe
```

Step 6: Execute the Pipe Program

```
None

./pipe
```

Expected Output:

```
None
Parent Process Started

Parent sent: Hello Child Process!


Child Process Started

Child received: Hello Child Process!

Child Process Completed

Parent Process Completed
```

The exact display order of parent and child messages may vary.

Step 7: Create the Shared Memory Program

```
None

nano shared_memory.c
```

Copy and paste Program 2.

Save the file:

- Press `Ctrl + O`.
- Press Enter.
- Press `Ctrl + X`.

Step 8: Compile the Shared Memory Program

```
None

gcc shared_memory.c -o shared_memory
```

Step 9: Execute the Shared Memory Program

```
None

./shared_memory
```

Expected Output:

```
None
Child Process Started

Child wrote data successfully


Parent Process Started

Parent received: Hello from Child using Shared Memory

Shared Memory Released

Parent Process Completed
```

Step 10: Verify IPC Resources

To inspect shared memory segments:

```
None
ipcs -m
```

To inspect message queues:

```
None
ipcs -q
```

To inspect semaphores:

```
None
ipcs -s
```

After executing the shared memory program, its segment should be removed automatically.

5. RESULT

Inter-Process Communication (IPC) mechanisms were successfully studied and implemented using C programming in Linux.

The following operations were demonstrated:

1. Created a communication channel using the `pipe()` system call.
2. Created parent and child processes using `fork()`.
3. Transferred messages between processes using `write()` and `read()`.
4. Created shared memory using `shmget()`.
5. Attached shared memory using `shmat()`.
6. Transferred data between parent and child processes through shared memory.
7. Used `wait()` for process synchronization.
8. Released shared memory resources using `shmdt()` and `shmctl()`.

Both Unix Pipes and Shared Memory were successfully demonstrated as IPC mechanisms.

6. CONCLUSION

In this practical, Inter-Process Communication (IPC) was studied and implemented using Unix Pipes and Shared Memory in the Linux environment. The pipe mechanism was used to transfer data between parent and child processes through a communication channel. Shared memory was used to exchange data by allowing multiple processes to access a common memory region. System calls such as `pipe()`, `fork()`, `read()`, `write()`, `shmget()`, `shmat()` and `shmctl()` were explored. Thus, the practical provided an understanding of process communication, data sharing and synchronization in operating systems.Practical 7 Completed

Next practical: Practical 8 – Deadlock Detection, Avoidance, and Recovery Using Banker's Algorithm.